

[Buscar Cursos](#)[Q \(Buscar Cursos\)](#)

Comenzado el	jueves, 23 de abril de 2020, 10:01
Estado	Finalizado
Finalizado en	jueves, 23 de abril de 2020, 11:00
Tiempo empleado	58 minutos 41 segundos
Puntos	10,70/20,00
Calificación	5,35 de 10,00 (54%)

Pregunta **1**

Correcta

Puntúa 1,00 sobre 1,00

Los atributos privados no son accesibles desde el objeto instancia, pero sí son accesibles desde la misma clase y las clases hijas

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

La respuesta correcta es 'Falso'

Pregunta **2**

Correcta

Puntúa 1,00 sobre 1,00

El patrón Singleton permite tener una única instancia de una clase, a la vez que permite que todas las clases tengan acceso a esa instancia

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **3**

Correcta

Puntúa 1,00 sobre 1,00

Es posible atrapar una excepción que sea de clase padre de una excepción declarada en un bloque catch().

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

La respuesta correcta es 'Falso'

Pregunta **4**

Correcta

Puntúa 1,00 sobre
1,00

Una variable de tipo Interface puede usarse como parámetro

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **5**

Finalizado

Puntúa 0,20 sobre 3,00

Considere una clase abstracta (Ordenable) que realice la ordenación de un arreglo compuesto por instancias de una clase de extienda de Ordenable:

```
public abstract class Ordenable
{
    public void ordenar_selección(Ordenable[] b) { .....}
    public abstract int compare(Ordenable b);
    public int p_menor(Ordenable[] b) { .....}
    public void intercambio( Ordenable[] b, int i, int j) {.....}
}
```

Indique esquematizando de la misma forma, cómo resolvería esto mismo utilizando una interface Comparable, que contenga el método abstracto compareTo(Object o).

Escriba también el método main() que realice la invocación.

```
public abstract class Ordenable implements Comparable
{
    public void ordenar_selección(Ordenable[] b) { .....}
    public int p_menor(Ordenable[] b) { .....}
    public void intercambio( Ordenable[] b, int i, int j) {.....}
}
```

```
public interface Comparable
{
    public int compareTo(Object o);
}
```

```
public class Caja extends Ordenable implements Comparable
{
    int tamano;

    public int getTamano(){....}

    public int compareTo(Caja caja){
        int rta = 0;
        if (this.tamano<caja.getTamano())
            rta =-1;
        else if (this.tamano>caja.getTamano())
            rta =1;
        }
        return rta;
    }

    public class prueba
    {
        public static void main(String[] args){
            Caja comparador;
            Caja cajas[20];

            comparador.ordenar_seleccion(cajas);
```

```
}
```

```
}
```

Comentario:

Regular, mezcló todo. No es necesario que Ordenable implemente Comparable, tampoco es necesario que Caja extienda de Ordenable.

Mal el parametro de ordenar_seleccion(). debió ser Comparable.

Pregunta **6**

Correcta

Puntúa 1,00 sobre
1,00

Una repercusión de utilizar atributos privados en una clase base es que, desde una clase extendida, se deben usar getters y setters para acceder a dichos atributos.

Seleccione una:

☒ Verdadero ✓

☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **7**

Finalizado

Puntúa 0,00 sobre 3,00

Indique qué debería considerar para que la clase Acorazado se pueda clonar y desarrolle el método Clone() para hacer una clonación correcta. Considerar que la clase Armamento_cañon y Armamento_Misil solo se pueden clonar si su posición es babor o estribos, pero nunca si están en popa o proa.

Barco

```
+String Nombre;  
+Float Velocidad;  
+Armamentos Armamento;
```

Armamento <abstracta>

```
+String Descripción;  
+Float Alcance;  
+enum Posiciones { POPA, BABOR, ESTRIBOR, PROA };
```

Armamento_Misil

```
+double Potencia_explosivo;  
+boolean ILS;  
+posicion Posiciones;
```

Armamento_Cañon

```
+int Cargador;  
+posicion Posiciones;
```

// la clase barco debe tener implementado su metodo clone()

```
public class Acorazado extends Barco{
```

```
clone()
```

```
{  
public Acorazado clone() throws CloneNotSupportedException {  
    return (Acorazado)super.clone();  
}  
}  
}  
  
Public class Armamento {  
  
public Armamento clone() throws CloneNotSupportedException {  
if Posiciones== babor || Posiciones== babo  
    return (armamento)super.clone()  
else throw new CloneNotSupportedException  
}  
  
}
```

Comentario: Acorazado no lo conozco.
Mal clonado Armamento.
Implementar interfaz Clonable

Pregunta 8

Correcta

Puntúa 1,00 sobre 1,00

El patrón Factory Method evita el uso del operador new() al momento de crear un objeto de un subtipo determinado.

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta 9

Correcta

Puntúa 1,00 sobre 1,00

Es posible atrapar una excepción que sea de tipo hija de una excepción declarada en un bloque catch(). Por eso es importante ordenar los bloques catch() según el tipo de excepción que atrape.

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **10**

Correcta

Puntúa 1,00 sobre 1,00

Un método que propaga excepciones si se sobrescribe puede cambiar las excepciones por otras cualquiera

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

La respuesta correcta es 'Falso'

Pregunta **11**

Correcta

Puntúa 1,00 sobre 1,00

Poner atributos privados en una clase base hace que desde una clase que la extienda no se pueda acceder directamente a los atributos

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **12**

Correcta

Puntúa 1,00 sobre 1,00

Sobrecargar significa tener dos o más métodos con el mismo nombre pero diferente lista de parámetros al menos

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta **13**

Incorrecta

Puntúa 0,00 sobre 1,00

Un bloque de inicialización estático se ejecuta cada vez que se crea un objeto

Seleccione una:

- ☒ Verdadero ✗
- ☐ Falso

La respuesta correcta es 'Falso'

Pregunta **14**

Finalizado

Puntúa 0,50 sobre 3,00

Considere el método siguiente y diseñe la excepción

```
Socio
+ String nombre;
+ int legajo;
+ Categoria categoria;
public int modificarCategoríaSocio(int legajo, Categoria nuevaCategoria) throws NoExisteSocioEception

{
    Socio aux = buscarSocio(legajo);
    if aux == null
        throw new NoExisteSocioEception(...);
    // código para realizar la modificación
}
```

Supongamos que en un cierto ámbito, la solución al problema que causa la excepción sea agregar un nuevo socio.

- Realice la clase NoExisteSocioException, completa.
- Determine qué información debería recibir el constructor de la Excepción.
- Realice un código try/catch en el ámbito supuesto, donde se invocaría el método modificarCategoríaSocio(...). Determine toda la información necesaria para poder solucionar el problema.
- Es necesario modificar el método modificarCategoríaSocio(...)?
- Justifique sus respuestas.

```
public class NoExisteSocioException extends Exception {
```

```
    public NoExisteSocioException(String arg0, int legajo) { // deberia recibir el legajo que no existe, ademas de un
mensaje con el hecho de que no existe
        super(arg0);
    }
}

try{
sociox.modificarCategoriaSocio(10,nuevaCategoria);
}
catch NoExisteSocioException {

}
```

Comentario:

la clase debe contener como atributo al menos lo que recibe como parámetro el constructor. Eso representa la información necesaria para utilizar en la zona de recuperación.

La zona catch no expresa ninguna solución.